國立臺北科技大學
National Taipei University of Technology

資料結構

Data Structure

Lab 06

姓名：簡紘田

學號：114310115

Lab01-Q1.(題目編號)

Please complete the following sections of the sample code:

- **Create a class named 'Queue' that includes the methods: 'enqueue', 'dequeue', 'isEmpty', and 'isFull'.**
- **Implement the 'initializeDeck' method.**
- **Implement the 'initializePlayer' method.**
- **Create the 'playerTurn' method.**
- **Create the 'dealerTurn' method.**

Code

```
#include <iostream>
#include <vector>
#include <string>
#include <algorithm>
#include <random>
#include <ctime>
using namespace std;
struct Player {// 玩家結構體,包含名稱和分數
    string name; // 玩家名稱
    int score; // 玩家分數
};
class Card {//建立一個 Card 類別,用來表示單張撲克牌
public:
    string suit; // 花色
    string rank; // 點數
    Card() {}
    Card(string s, string r) : suit(s), rank(r) {}
    void display() const {
        cout << rank << " of " << suit << endl;
    }
};
class Deck {// 牌組類別,負責生成和洗牌
private:
    Card cards[52]; // 52 張牌
    int index; // 目前發牌索引
public:
    Deck() : index(0) {// 初始化牌組
```

```
        string suits[] = { "Hearts", "Diamonds", "Clubs", "Spades" };// 四種花色
        string ranks[] = { "2", "3", "4", "5", "6", "7", "8", "9", "10", "J", "Q",
"K", "A" };// 13 張牌
        int k = 0;
        for (int i = 0; i < 4; i++) { // 4 種花色
            for (int j = 0; j < 13; j++) { // 13 種點數
                cards[k] = Card(suits[i], ranks[j]);
                k++;
            }
        }
    }
    void shuffleDeck() {// 洗牌
        srand(time(0));
        for (int i = 51; i > 0; i--) {// 從最後一張牌開始，向前遍歷
            int j = rand() % (i + 1);// 生成隨機索引 j
            swap(cards[i], cards[j]);// 交換牌組中的兩張牌
        }
    }
    // 取得指定位置的牌
    Card getCard(int i) const {
        return cards[i];
    }
};

// Queue 類別
class Queue {
private:
    vector<Card> deck; // 用來存放卡牌的容器
    int Front;         // 指向牌堆的前端
    int Rear;          // 指向牌堆的尾端
    int capacity;      // 容器最大容量
public:
    // 建構函式
    Queue(int size) : Front(-1), Rear(-1), capacity(size) {}
    // 插入元素到佇列尾端
    void enqueue(Card card) {
        if (isFull()) { // 檢查是否滿
            cout << "牌堆已滿,無法插入!" << endl;
```

```
            return;
        }

        if (isEmpty()) {
            Front = 0;   // deck 裡面第一張的位置有牌了
        }

        deck.push_back(card); // 插入元素到尾端
        Rear++; // 尾端指標移動
    }
    // 從佇列中移除最前端元素
    void dequeue() {
        if (isEmpty()) {    // 檢查是否空
            cout << "牌堆為空，無法刪除！" << endl;
            return;
        }

        Front++;    //將下一張卡牌設為牌組起點，原來的排頭已被抽走

        if (Front > Rear) { // 最後一張牌被抽走後將重置為空排組
            Front = -1;
            Rear = -1;
            deck.clear();
        }
    }
    Card front() {// 獲取佇列最前端元素
        if (isEmpty()) {// 檢查是否空
            cout << "牌堆為空！" << endl;
            return Card("", "");
        }
        return deck[Front];
    }

    bool isEmpty() const {// 判斷佇列是否為空
        return (Front == -1 && Rear == -1);
    }

    bool isFull() const {// 判斷佇列是否已滿
```

```
        return deck.size() >= capacity;
    }
};

void initializeDeck(Queue& cardDeck) {// 初始化並洗牌
    Deck deck;
    cout << "初始牌堆:\n";
    for (int i = 0; i < 52; i++) {
        deck.getCard(i).display();//印出原始牌堆
    }
    deck.shuffleDeck();//洗牌
    cout << "\n洗牌後:\n";
    for (int i = 0; i < 52; i++) {
        deck.getCard(i).display();//印出洗牌後的牌堆
    }

    for (int i = 0; i < 52; i++) {//將洗好的牌放入牌堆
        cardDeck.enqueue(deck.getCard(i));
    }
}

void initializePlayer(Player* player, string name, Queue& cardDeck) {// 初始化玩家,發兩張牌
    player->name = name;    // 玩家名稱
    player->score = 0;  // 玩家分數
    cout << player->name << " 抽到: ";

    for (int i = 0; i < 2; i++) {   // 發兩張牌
        Card drawnCard = cardDeck.front(); // 將最前面的卡牌複製出來
        cardDeck.dequeue(); // 將牌頭"刪除"

        //將抽到的牌印出
        cout << drawnCard.rank << " of " << drawnCard.suit << "  ";

        if (drawnCard.rank == "A")
            player->score += 1;     // 計算 A 分數
        else if (drawnCard.rank == "J" || drawnCard.rank == "Q" || drawnCard.rank == "K")
```

```
            player->score += 10;    // 計算 JQK 分數
        else
            player->score += stoi(drawnCard.rank);  // 計算 2-10 分數
    }
    cout << "\n";
    cout << player->name << " 玩家的初始分數: " << player->score <<
        endl;   // 印出玩家的初始分數
}

void playerTurn(Player* player, Queue& cardDeck) {  // 玩家回合
    char choice;

    while (player->score < 21 && !cardDeck.isEmpty()) { // 玩家小於 21 點且牌堆不為
空
        cout << player->name << " 您的手牌分數目前為: " << player->score
            << " 要抽牌嗎?(h = 抽, s = 停) : ";//顯示選擇
        cin >> choice;//輸入選擇

        if (choice == 'h') {    // 如果玩家選擇抽排
            Card newCard = cardDeck.front();    // 將牌頭複製到 newCard 上面
            cardDeck.dequeue(); // 將牌頭"刪除"

            if (newCard.rank == "A") player->score += 1;
            else if (newCard.rank == "J" || newCard.rank == "Q" || newCard.rank ==
"K")
                player->score += 10;
            else
                player->score += stoi(newCard.rank);

            cout << "抽到: " << newCard.rank << " of " << newCard.suit
                << " 總分: " << player->score << endl;

            if (player->score > 21) {
                cout << "爆掉！\n";
                return;
            }
        }
        else if (choice == 's') {//如果玩家選擇停牌
```

```
            cout << player->name << " 選擇停牌,總分: " << player->score
                << endl;//印出總點數
            break;
        }
        else {
            cout << "請輸入有效選項 (h = 抽, s = 停)!" << endl;  // 輸入無效
        }
    }
}

void dealerTurn(Player* dealer, Queue& cardDeck) {// 莊家回合
    while (dealer->score < 17 && !cardDeck.isEmpty()) {// 莊家小於 17 點且牌堆不為
空
        Card newCard = cardDeck.front();    // 將牌頭複製到newCard 上面
        cardDeck.dequeue(); // 將牌頭"刪除"

        if (newCard.rank == "A") dealer->score += 1;//計算 A 點數
        else if (newCard.rank == "J" || newCard.rank == "Q" ||
            newCard.rank == "K") dealer->score += 10;//計算 JQK 點數
        else dealer->score += stoi(newCard.rank);//計算 2-10 點數
        //印出抽到的牌和總點數
        cout << "莊家抽到: " << newCard.rank << " of " << newCard.suit
            << " 莊家目前總分: " << dealer->score << endl;
        if (dealer->score > 21) {//如果莊家大於 21 點
            cout << "莊家爆了!玩家獲勝!" << endl;//玩家獲勝
            return;
        }
    }
}

// 判斷勝負
void determineWinner(Player* player, Player* dealer) {
    if (player->score > 21)
        cout << player->name << " 爆了!莊家獲勝!\n";//玩家爆牌,莊家獲勝
    else if (dealer->score > 21 || player->score > dealer->score)
        cout << player->name << " 贏了!\n";//莊家爆牌或玩家點數大於莊家, 玩家獲勝
    else if (player->score == dealer->score)
        cout << "平手!\n";//莊家與玩家點數相同, 平手
```

```
    else
        cout << "莊家贏了!\n";
}

int main() {
    srand(time(0));
    Queue cardDeck(52);//宣告牌堆
    initializeDeck(cardDeck);//初始化牌堆

    Player player, dealer;//宣告並初始化莊家以及玩家
    initializePlayer(&player, "玩家", cardDeck);
    initializePlayer(&dealer, "莊家", cardDeck);

    playerTurn(&player, cardDeck);
    if (player.score <= 21) {
        cout << "\n 莊家回合...\n";
        dealerTurn(&dealer, cardDeck);
        determineWinner(&player, &dealer);
    }
    return 0;
}
```

Result

D:\紘田\大學\資料結構\Lab06

```
初始牌堆:
2 of Hearts
3 of Hearts
4 of Hearts
5 of Hearts
6 of Hearts
7 of Hearts
8 of Hearts
9 of Hearts
10 of Hearts
J of Hearts
Q of Hearts
K of Hearts
A of Hearts
2 of Diamonds
3 of Diamonds
4 of Diamonds
5 of Diamonds
6 of Diamonds
7 of Diamonds
8 of Diamonds
9 of Diamonds
10 of Diamonds
J of Diamonds
Q of Diamonds
K of Diamonds
A of Diamonds
2 of Clubs
3 of Clubs
4 of Clubs
5 of Clubs
6 of Clubs
7 of Clubs
8 of Clubs
9 of Clubs
10 of Clubs
J of Clubs
Q of Clubs
K of Clubs
A of Clubs
2 of Spades
3 of Spades
4 of Spades
5 of Spades
6 of Spades
7 of Spades
8 of Spades
9 of Spades
10 of Spades
J of Spades
Q of Spades
K of Spades
A of Spades
```

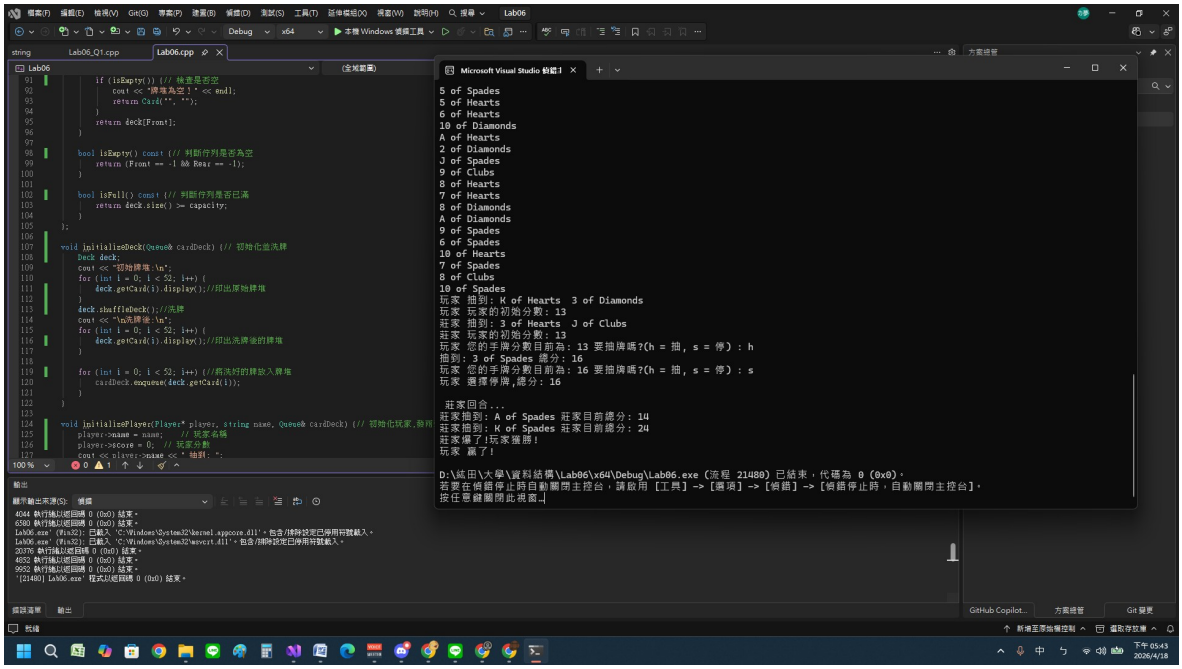
D:\紘田\大學\資料結構\Lab06'

```
洗牌後:
K of Hearts
3 of Diamonds
3 of Hearts
J of Clubs
3 of Spades
A of Spades
K of Spades
Q of Spades
4 of Diamonds
Q of Clubs
J of Diamonds
2 of Hearts
2 of Spades
J of Hearts
A of Clubs
6 of Clubs
K of Diamonds
5 of Diamonds
9 of Hearts
2 of Clubs
10 of Clubs
9 of Diamonds
6 of Diamonds
3 of Clubs
5 of Clubs
4 of Spades
8 of Spades
7 of Diamonds
Q of Diamonds
K of Clubs
4 of Clubs
Q of Hearts
7 of Clubs
4 of Hearts
5 of Spades
5 of Hearts
6 of Hearts
10 of Diamonds
A of Hearts
2 of Diamonds
J of Spades
9 of Clubs
8 of Hearts
7 of Hearts
8 of Diamonds
A of Diamonds
9 of Spades
6 of Spades
10 of Hearts
7 of Spades
8 of Clubs
10 of Spades
```

```
玩家 抽到: K of Hearts  3 of Diamonds
玩家 玩家的初始分數: 13
莊家 抽到: 3 of Hearts  J of Clubs
莊家 玩家的初始分數: 13
玩家 您的手牌分數目前為: 13 要抽牌嗎?(h = 抽, s = 停) : h
抽到: 3 of Spades 總分: 16
玩家 您的手牌分數目前為: 16 要抽牌嗎?(h = 抽, s = 停) : s
玩家 選擇停牌,總分: 16

 莊家回合...
莊家抽到: A of Spades 莊家目前總分: 14
莊家抽到: K of Spades 莊家目前總分: 24
莊家爆了!玩家獲勝!
玩家 贏了!

D:\紘田\大學\資料結構\Lab06\x64\Debug\Lab06.exe (流程 21480) 已結
束，代碼為 0 (0x0)。
若要在偵錯停止時自動關閉主控台，請啟用 [工具] -> [選項] -> [偵錯]
 -> [偵錯停止時，自動關閉主控台]。
按任意鍵關閉此視窗...
```



Discussion

在我的觀察下，stack 跟 queue 在這次的題目使用情況來說，對於使用者在遊玩 21 點遊戲本體上沒有什麼差別，但從一副牌的形成、洗牌到抽卡來說 stack 更符合現實遊玩邏輯。

先是 for 從 2 填入直到 A 填完，就是一張一張將 2 到 A 放到牌桌上的待抽區所以 2 在最底 A 在最頂，再來從上面的 A 開始抽一張與整副牌之中的隨機一張交換，重複動作 52 次，得到一副被打亂的新牌組，最底為牌頭，最頂為牌尾，而抽卡肯定是由最頂部(牌尾)的那張開始抽，也就是先進入待抽區的最晚會被抽出，所以我覺得更符合 stack 的型態。